

Simulation Integration & Troubleshooting Report

Project: PX4 SITL / Gazebo Drone Simulation Environment

Status: Software-In-The-Loop (SITL) Cross-Network Pipeline Resolved

Current Location Focus: Taramani, Chennai (Host) / Baylands, California (Simulation World)

Executive Summary

Day 4 was dedicated to resolving cross-network isolation bugs between the **WSL2 Ubuntu Linux Subsystem** (where the PX4 Autopilot engine and Gazebo Sim reside) and the **Windows Host OS** (running the QGroundControl Ground Control Station interface).

Because WSL2 acts as a virtualized router with a dynamic subnet, standard local loopback (127.0.0.1) communication failed initially. Through sequential troubleshooting of firewall pathways, environment pipelines, and parameter blocks, we successfully bridged the gap to establish a direct telemetry feed.

Problems Encountered & Technical Analysis

1. WSL2 Network Isolation:

The PX4 simulation defaulted to restricted local telemetry loops. Windows network protocols viewed incoming WSL2 data packets as external unauthenticated traffic, causing QGroundControl to display a persistent Disconnected status.

2. Localhost Variable Mismatch:

Setting network routes inside secondary terminal tabs failed to propagate to the main simulation environment because Linux terminal variables are tightly isolated to their parent shell process.

3. Internal Autopilot MAVLink Block:

The autopilot firmware defaulted to `MAV_0_BROADCAST = 0`, locking all flight telemetry data strictly inside the Linux kernel loopback interface.

Action Items & Implemented Solutions

Phase	Action Taken	Operational Impact
1. Port Configuration	Adjusted Windows netsh interface portproxy rules to mirror listenport=14550 explicitly to the loopback adapter interface.	Opened a dedicated, bidirectional network pipeline between the host OS and the subsystem.
2. Pipeline Binding	Identified the exact virtual ethernet interface (172.29.48.1) and initiated path binding inside the local build shell directory.	Ensured simulation packets were directly aimed at the Windows monitoring station.
3. Hard-Coded Port Pass	Initiated the PX4 flight simulator target by embedding the port override directly into the native runtime argument string (__port:=14550).	Bypassed volatile system environment variables completely for consistent initialization.
4. Parameter Override	Injected the param set MAV_0_BROADCAST 1 command directly into the active drone runtime controller terminal (pxh>).	Unlocked the telemetry broadcast layer, pushing packet updates through the network bridge.

Final System State

- **Autopilot Environment:** Fully stable inside ~/robotics_sim/PX4-Autopilot/.
- **Network Status:** Connected. QGroundControl automatically clears preflight safety checks (Preflight check: PASSED) the moment the MAVLink broadcast is engaged.
- **Flight Readiness:** The system is primed at the pxh> shell prompt, standing by to execute automated commander takeoff instructions.